Early Bird 时序攻击：在 EDR 钩子落地前执行

上章的 APC 注入有个致命局限：无法控制目标线程何时进入“可提醒状态”。 如果目标线程一直在忙，APC 永远不会执行。

Early Bird 解决了这个问题：它创建个“挂起”的新进程，在进程还没开始运行、EDR 钩子还没完全落地时，就把 APC 塞进主线程。恢复线程后 Shellcode 会在进程入口点之前执行。

- **核心概念：为什么会有“时序窗口”**

➢ 进程创建的物理时序

当调用 CreateProcess 时，Windows 内核做的事情按顺序是：

1. 创建进程对象（EPROCESS）
2. 创建主线程对象（ETHREAD），状态为挂起
3. 加载 ntdll.dll（最先加载，因为要调用它初始化）
4. 加载 kernel32.dll、user32.dll 等
5. 加载 EDR 的 DLL（比如某杀软的 hook dll）
6. EDR 的 DLL 在 ntdll 的 API 入口处写入跳转指令（Hook 落地）
7. 主线程开始执行，进入入口点

关键点：在⑤和⑥之间有个极短的时间窗口——DLL 已经加载，但 EDR 的 Hook 没写完。

➢ **CREATE_SUSPENDED 的作用**

CreateProcess 的第六个参数是 dwCreationFlags。如果传入 CREATE_SUSPENDED：

1. 进程和主线程都创建好了。
2. 但主线程处于挂起状态，CPU 不会执行它的任何一条指令。
3. 此时，进程的 DLL 已经加载到④或⑤，但 EDR 的 Hook 可能还没落地（取决于 EDR 的初始化时机）。

物理意义：拿到了个“活的但还没开始跑”的进程。它的内存里，ntdll.dll 的 API 入口可能还是干净的（没有被 EDR 插入跳转指令）。

➢ **Early Bird 的执行时机**

```
CreateProcess(CREATE_SUSPENDED)
        ↓
进程创建，主线程挂起
        ↓
【窗口期】EDR 钩子可能还没落地
        ↓
VirtualAllocEx + WriteProcessMemory 写入 Shellcode
        ↓
NtQueueApcThread 向主线程 APC 队列插入 Shellcode
        ↓
ResumeThread 恢复主线程
        ↓
主线程从挂起点恢复，系统检查 APC 队列
```

↓

APC 被执行（在进程入口点之前!）

↓

Shellcode 执行完毕，继续进程正常初始化

问：为什么 APC 会在入口点之前执行?

答：因为主线程在挂起恢复后，会先检查 APC 队列。APC 队列里有待执行的回调，系统会优先执行它，然后才继续走进程的初始化流程。这就是 Early Bird 的“抢跑”本质。

- **三种注入方式对比**

对比项	CreateRemoteThread	普通 APC	Early Bird
是否创建新线程	是	否	否
执行时机	随时	依赖目标线程进入可提醒状态	进程入口点之前
EDR 钩子是否已落地	是	是	可能未落地
任务管理器显示	目标进程多一个线程	无新线程	无新线程
EDR 检测难度	低（远程线程监控）	中（APC 监控）	高（在 Hook 之前执行）

- **代码示例（见文件夹）**

➢ **第一版：明文 API 实现**

物理动作解释:

步骤	物理意义
CREATE_SUSPENDED	主线程挂起，进程已加载 DLL，但还没开始执行
VirtualAllocEx + WriteProcessMemory	在目标进程写 Shellcode（此时 EDR 钩子可能还没落地）
NtQueueApcThread	把 Shellcode 插入主线程 APC 队列
ResumeThread	主线程恢复，先执行 APC，再走入口点

➢ **第二版：无 IAT 版本**

解释略 ……

- **为什么 Early Bird 能绕过 EDR 钩子**

➢ **EDR 钩子的落地时序**

EDR 的用户态钩子通常是通过以下方式实现的：

- 注入一个 DLL 到目标进程。
- 在该 DLL 的 DllMain 里，修改 ntdll.dll 的 API 入口，写入跳转指令。

关键：这个 DLL 的 DllMain 是在进程初始化后期才被调用的。在 CREATE_SUSPENDED 状态下，DllMain 可能还没执行完，钩子还没完全落地。

➢ **Early Bird 的抢跑逻辑**

创建挂起进程 → EDR DLL 加载但 DllMain 未执行 → 钩子未落地

↓

写入 Shellcode + 插入 APC（此时 ntdll 的 API 还是干净的）

↓

恢复线程 → APC 执行 → Shellcode 跑在钩子落地之前

结果：Shellcode 在 EDR 的“监控视野”之外执行。

- **观察三种方式的差异**

用任务管理器（详细信息视图）观察：

方式	目标进程的线程数	EDR 告警概率
CreateRemoteThread	多一个线程	高
普通 APC	线程数不变	中
Early Bird	线程数不变，执行在入口点前	低

- **本课小结**

- Early Bird 的核心：利用 CREATE_SUSPENDED 的时序窗口，在 EDR 钩子落地前执行 Shellcode。
- 流程：挂起创建 → 写 Shellcode → 插 APC → 恢复线程。
- 优势：不创建新线程，执行在入口点之前，避开 EDR 钩子。
- 局限：依赖 EDR 钩子的落地时序，不是所有 EDR 都有这个窗口。